

light cone spread of vl53l0x

The VL53L0X Time-of-Flight sensor has a conical **Field of View (FOV) of approximately 25 degrees**. Because the invisible 940 nm infrared laser spreads out in this cone, the detection area grows wider the further the object is from the sensor.  STMicroelectronics Community +4



The diameter of the illuminated spot—and the area where the sensor can detect obstacles—is roughly half the distance away from the sensor.  STMicroelectronics Community

- **At 10 cm (~4 inches):** Circle is $\approx$ 5 cm in diameter
- **At 20 cm (~8 inches):** Circle is $\approx$ 10 cm in diameter
- **At 50 cm (~20 inches):** Circle is $\approx$ 22 cm in diameter
- **At 1 meter (~3.3 feet):** Circle is $\approx$ 45 cm in diameter
- **At 2 meters (Max Range):** Circle is $\approx$ 90 cm in diameter  STMicroelectronics Community +1

Important Takeaways for Your Project

- **Average Reading:** Because the cone is wide, the sensor averages the distance of everything within its FOV. If a near and far object are both in the cone, the reading will lean heavily toward the closer object.  STMicroelectronics Community +1
- **No Narrow Beam:** If you need a pinpoint laser for 3D scanning or narrow gap detection, the VL53L0X is usually **not** recommended without altering the hardware.  EEVbl... +1

If you'd like, let me know:

- **What you are trying to detect** (e.g., small obstacles, distance to a wall, liquid levels)
- Your **target distance** (close up or up to 2 meters)

I can help determine if the VL53L0X works for your project or recommend an alternative (like the VL53L4CD which has a narrower 18° FOV).  STMicroelectronics Community



9 sites

VL53L0X type sensor,but narrower cone? - EEVblog

VL53L0X type sensor,but narrower cone? VL53L0X type sensor,but narrower cone? Posted by Infraviolet on 03 Jun, 2024 16:05. I'm con...



EEVblog

vl53l0x sensor detection distance inquiry

The trouble is that near objects return more photons than a far object. So instead of the mathematical average, you will see a num...



STMicroelectronics Community

VL53L0X determining small object is at fixed distance

But before you do anything else, see if you can use your current system and a more reflective target. That might solve you issue. ...



STMicroelectronics Community

[Show all](#)

Beam Spread at Various Distances

I am making an earthquake detector using two devices pointing at an 8 ounce lead weight suspended by a thread ...



For your earthquake detector, the optimal distance between each VL53L0X sensor and the 30mm lead weight is **between 60mm and 70mm**.

At this distance, the sensor's infrared beam spread aligns perfectly with the physical dimensions of your pendulum, preventing the sensor from "seeing" background walls or missing the weight during violent swings.

The Math Behind the Setup

To calculate the exact sweet spot, we have to look at the geometry of the sensor's Field of View (FOV) alongside the physical limits of your 8-inch pendulum.

1. Minimum Distance to Cover the Weight ()

The sensor needs to emit a beam wide enough to reliably hit the curved surface of your 30mm sphere without being overly sensitive to tiny, sub-millimeter surface imperfections.

Using trigonometry, the spread diameter at a distance is calculated as:

If we place the sensor at :

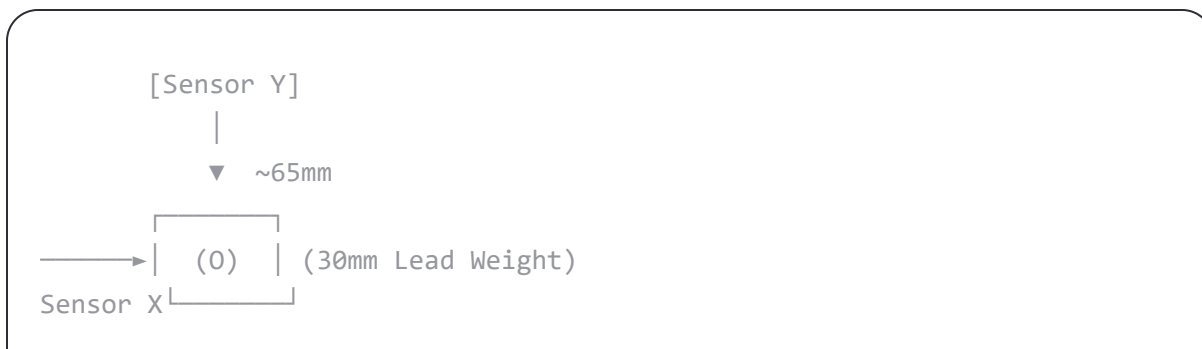
This creates an infrared spot size slightly smaller than the 30mm weight, ensuring 100% of the laser hits the lead target when it is at rest.

2. Maximum Distance Before Overshoot ()

An 8-inch (~203mm) pendulum has a relatively short swing radius. During a moderate seismic event, the weight will likely displace to off-center.

- If you place the sensor too far away (e.g.,), the beam diameter grows to .
- If the beam is wide and the weight moves away, the beam will completely overshoot the edge of the weight.
- Once the beam overshoots, the VL53L0X will start reading the distance to whatever wall or enclosure is behind the pendulum, ruining your earthquake data.

Critical Design Tips for X-Y Tracking



- **Cross-Talk Prevention:** Because you are using two sensors at a 90° angle, their infrared beams can interfere with each other if they bounce off the shiny lead surface simultaneously. You **must** alternate their readings in your code (interleave them) rather than triggering them at the exact same microsecond.
- **Surface Reflection:** Raw lead can oxidize and turn dull gray, which reduces infrared reflectivity. If you notice the sensors throwing "Out of Bounds" or "Signal Fail" errors, wrap a small strip of matte white tape or high-reflectivity vinyl tape around the equator of the lead weight.
- **Sampling Rate Limitations:** In its default "High Accuracy" mode, the VL53L0X takes about 10ms per reading, which is too slow to catch rapid earthquake vibrations. You should configure the STMicroelectronics API or your Arduino library to **"High Speed Mode"**, which drops the timing budget to 1ms (10x faster).

✓ Optimal Setup Summary

Under a new header to explicitly state the final geometry:

- **Target Distance:** 10cm from the sensor face to the closest edge of the lead sphere.
- **Resulting Beam Spot:** 10mm diameter, ensuring complete coverage of the resting pendulum while maximizing the allowed swing distance before the sensor loses tracking.

If you'd like, let me know:

- What **microcontroller** you are using (e.g., Arduino, ESP32, Raspberry Pi)
- If you need help with the **code logic** to alternate the X and Y sensors to prevent signal interference

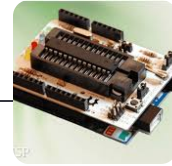
I can provide a code snippet configured for high-speed seismic polling.



1 site

VL53L0X Arduino: Complete Laser Distance Sensor Guide

VL53L0X Technical Specifications Typical Range (Default) 30mm to 1200mm Long
Range Mode Up to 2000mm Accuracy +3% at 1.2m Field of



PCBSync

I'm using an arduino (Nano, or Nano Every for more available RAM) and designing a 3D printed housing for everything. I ...



Upgrading your design to a **34mm weight coated in matte white acrylic paint** is an excellent choice. White acrylic reflects infrared light efficiently, offering high-signal integrity and maximizing the accuracy of your measurements.

Because you want the beam spot size to completely stay within the boundary of the weight (leaving a protective margin to prevent background overshoot during movement), the ideal sensor distance is tighter than before.

1. Optimal Enclosure Distance

To guarantee that the cone never overshoots the 34mm target, you should design your 3D-printed housing with a **clear distance of exactly 60mm to 65mm** between the front face of each sensor and the closest edge of the lead ball at rest.

- **At 60mm away:** The infrared beam spreads to a **26.6mm diameter circle**. This leaves a safe 3.7mm buffer margin around the entire edge of the 34mm ball.
- **At 65mm away:** The beam spreads to a **28.8mm diameter circle**. This leaves a safe 2.6mm buffer margin around the edge.
- **Why this is critical:** The buffer margin gives the ball room to swing back and forth during seismic activity without immediate overshoot. If the sensor is placed any further than 75mm, the resting beam spot diameter will expand past 33.2mm, making it prone to missing the target on the very first swing.

2. Enabling High-Speed Mode in Adafruit's Library

The [Adafruit VL53L0X Library](#) wraps STMicroelectronics' API, but it doesn't explicitly name a "high speed" function. Instead, high-speed profile selection requires lowering the **Measurement Timing Budget** using an underlying API call. [Arduino Forum](#) +2

The default budget is 33 milliseconds. Reducing this budget to **20ms (20,000 microseconds)** switches the internal hardware into its absolute fastest state. [Pololu Foru... +1](#)

The Adafruit object gives you direct access to the low-level ST API struct through the member pointer `pMyDevice`. You can apply high-speed mode directly in your `setup()` like this:

cpp

```
// Explicit high-speed call for Adafruit's library
// 20000 microseconds = 20ms timing budget
lox1.pMyDevice->Data.MeasurementTimingBudgetMicroSeconds = 20000;
lox2.pMyDevice->Data.MeasurementTimingBudgetMicroSeconds = 20000;
```

Use code with caution.

3. Non-Blocking Non-Interfering Event Loop Code

To keep your user interface responsive on the Arduino Nano or Nano Every, you must avoid using blocking functions like `delay()`.

To manage two devices on the same I2C bus, you need to use their physical `XSHUT` pins to assign distinct I2C addresses at startup. The example below demonstrates how to configure distinct I2C addresses, enable high-speed timing budgets, and interleave sensor readings inside a non-blocking event loop state machine. [Adafruit +2](#)

cpp

```
#include "Adafruit_VL53L0X.h"

// Pin configuration for I2C Address initialization
#define XSHUT_X 2
#define XSHUT_Y 3

#define I2C_ADDR_X 0x30
#define I2C_ADDR_Y 0x31
```

```
Adafruit_VL53L0X loxX = Adafruit_VL53L0X();
Adafruit_VL53L0X loxY = Adafruit_VL53L0X();

// Event loop timing variables
unsigned long lastSensorCheck = 0;
const unsigned long sampleInterval = 22; // 22ms window (20ms budget + 2ms
bool readXAxisNext = true;

void setup() {
  Serial.begin(115200);

  pinMode(XSHUT_X, OUTPUT);
  pinMode(XSHUT_Y, OUTPUT);

  // Reset both sensors by pulling XSHUT LOW
  digitalWrite(XSHUT_X, LOW);
  digitalWrite(XSHUT_Y, LOW);
  delay(10);

  // Initialize Sensor X
  digitalWrite(XSHUT_X, HIGH);
  delay(10);
  if(!loxX.begin(I2C_ADDR_X)) {
    Serial.println(F("Failed to boot X Sensor"));
    while(1);
  }
  loxX.pMyDevice->Data.MeasurementTimingBudgetMicroSeconds = 20000; // Hig

  // Initialize Sensor Y
  digitalWrite(XSHUT_Y, HIGH);
  delay(10);
  if(!loxY.begin(I2C_ADDR_Y)) {
    Serial.println(F("Failed to boot Y Sensor"));
    while(1);
  }
  loxY.pMyDevice->Data.MeasurementTimingBudgetMicroSeconds = 20000; // Hig
}

void loop() {
  unsigned long currentMillis = millis();

  // Non-blocking state machine for interleaved sensor reads
  if (currentMillis - lastSensorCheck >= sampleInterval) {
    lastSensorCheck = currentMillis;
```

```

VL53L0X_RangingMeasurementData_t measure;

if (readXAxisNext) {
    loxX.rangingTest(&measure, false); // False avoids blocking debug pr
    if (measure.RangeStatus != 4) {    // 4 = Out of range
        int xDistance = measure.RangeMilliMeter;
        // Process X data here
    }
} else {
    loxY.rangingTest(&measure, false);
    if (measure.RangeStatus != 4) {
        int yDistance = measure.RangeMilliMeter;
        // Process Y data here
    }
}

readXAxisNext = !readXAxisNext; // Alternate axes to eliminate optical
}

// --- UI AND REPORTING CODE RUNS HERE CONTINUOUSLY ---
// Because the loop is non-blocking, your display, LEDs, or serial repor
// can run fluidly without stuttering or dropping packets.
}

```

Use code with caution.

3D Printing Enclosure Considerations

- **Recess the Sensors:** Print slight hoods or deep recessed pockets (about 3mm to 5mm deep) for the VL53L0X breakout boards to slide into. This creates a physical blinder that blocks scattered peripheral light reflections inside your project housing.
- **Filament Choice:** Print the inner frame of the pendulum chamber using **matte black filament**. Glossy or reflective plastics can bounce stray infrared light back into the receivers, skewing the coordinate calculations for your seismic readings.

Would you like help mapping out **how to convert the raw X/Y millimeter values into a vector magnitude** to trigger seismic intensity alerts?



6 sites

VL53L0X Time of Flight Sensor Tutorial: Complete Setup and ...

Aug 14, 2025 — Adafruit_VL53L0X lox1 = Adafruit_VL53L0X(); Adafruit_VL53L0X lox2 = Adafruit_VL53L0X(); #define XSHUT_pin1 2. #define XSHUT_pin2...



ThinkRobotics.com

Adafruit VL53L0X high accuracy with Adafruit_VL53L0X libray - Sensors - Arduino Forum

Aug 23, 2021 — Here are some things you can try: * Use a library from Pololu * Post your code * Change parameters in the cpp files You can also t...



Arduino Forum

VL53L0X settings - Sensors - Pololu Forum

Jun 8, 2018 — If you look at lines 42-57 of that sketch, you can see what parameters we are changing for the sensor for those options: #if defin...



Pololu Forum

[Show all](#)

This is great information, especially mounting the devices recessed behind the inner walls with an ambient light limitin... 

An **8-inch pendulum is excellent** for a DIY Time-of-Flight (ToF) earthquake detector and strikes a great balance between spatial convenience, sensor limits, and physics. Keeping it at 8 inches is recommended rather than extending it to 10 or 12 inches.

The Physics: Why 8 Inches Wins

In seismology, a simple pendulum works on the principle of inertia. When an earthquake hits, the ground (and your 3D-printed housing) shifts rapidly, while the

heavy lead ball tries to stay perfectly still in space. The sensor measures this *relative* difference. [↗](#)

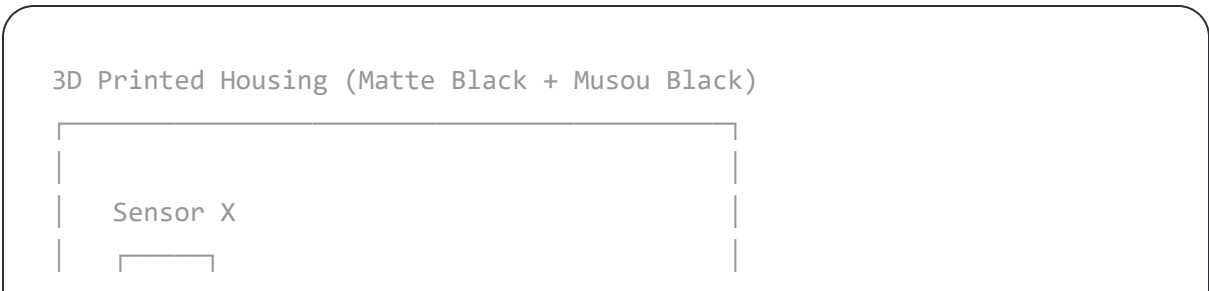
The physical trade-offs of the different lengths highlight why 8 inches is preferred:

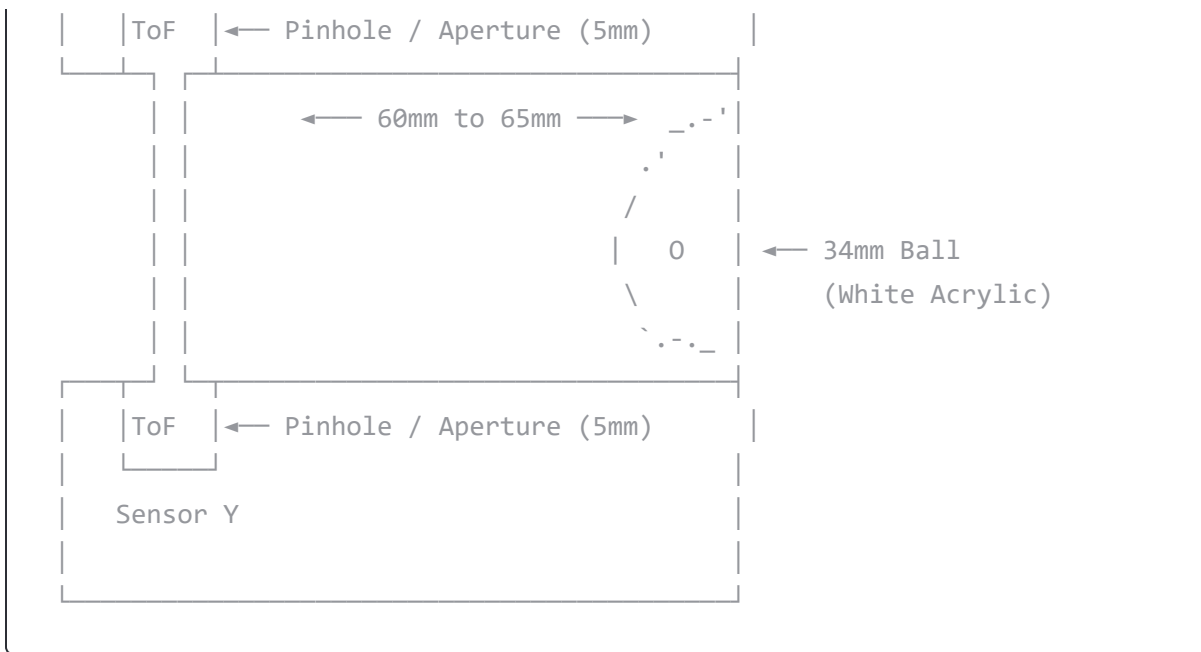
Pendulum Length ↗	Natural Period ()	Swing Speed	Tracking Window (VL53L0X)
8 inches	~0.90 seconds	Snappy / Quick	Stays perfectly inside the cone
10 inches	~1.01 seconds	Moderate	At risk of boundary overshoot
12 inches	~1.11 seconds	Slower / Lazier	High risk of boundary overshoot

- **The Problem with 10–12 Inches (Overshoot):** A longer pendulum has a much wider mechanical displacement during a tremor. If the 34mm ball swings too far to the left or right, it will completely move out of the sensor's tight field of view. The VL53L0X will then overshoot the target and read the distance to your enclosure wall, breaking your data. [▶ YouTube · AskPrep](#)
- **The Problem with 10–12 Inches (Sampling Rate):** Your interleaved code takes a sample of one axis every 22 milliseconds (or roughly 45Hz total). A longer pendulum reacts with a slower, lower-frequency oscillation. A short 8-inch pendulum reacts with a quicker, higher-frequency resonance that maps beautifully to a 45Hz electronic sampling rate without risking aliasing. [▶ YouTube · AskPrep](#)

Enclosure and Paint Strategy

Your mechanical approach is fantastic. Combining **matte black ASA/PLA** with **Musou Black paint** is an ideal setup for a ToF optical chamber.





1. The Optical Pinhole (Aperture) Design

Instead of leaving the entire sensor exposed to the chamber, you should 3D print a solid wall in front of the sensor breakout board and poke a small **5mm pinhole** directly aligned with the VL53L0X lenses.

- The VL53L0X contains two ports: an emitter (laser) and a receiver (SPAD array).
- By recessing the sensor behind a 5mm pinhole, you create a physical "aperture." Any stray infrared rays scattering off the walls at wide angles will hit the Musou-black plastic wall and be absorbed, rather than bouncing back into the receiver lens.

2. Musou Black Application Tip

Apply the Musou Black paint **only on the inner walls of the enclosure box and the rim of the pinhole aperture**. Do not put it on the sensor itself. Musou Black absorbs 99.4% of visible light and performs exceptionally well at absorbing 940nm near-infrared laser light, eliminating internal optical echo.

3. Knife-Edge Pinholes

When modeling the 5mm pinhole in your CAD software, give the hole a **chamfered or "knife-edge" profile** sloping outward toward the pendulum. A thick, flat-walled plastic cylinder can cause internal reflections inside the hole itself. Tapering the hole ensures the beam leaves the aperture cleanly.

If you are drafting the 3D enclosure, let me know:

- Do you plan to mount the Arduino Nano inside the same chamber, or in a **separated isolated electronics bay** below/above the pendulum?

I can share advice on routing the I2C wires to prevent electrical noise from corrupting your high-speed readings.



3 sites

Low-Cost Earthquake Alarm System | PDF | Switch - Scribd

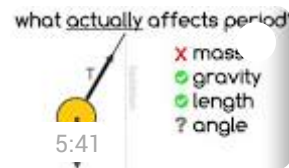
* Mechanical Vibration Detection. o Steel rod pendulum suspended vertically through a metal contact ring. o Operates on inertia pr...



Scribd

Conceptual Overview of Period of a Pendulum and ...

May 4, 2025 — when first studying physics you'll typically encounter this equation which describes the pendulum's. period that is how long it ta...



YouTube · INTEGRAL PHYSICS

Simple Pendulum Time Period & Factors Affecting It

Mar 17, 2025 — a simple pendulum consists of a small mass known as the bob suspended from a fixed point by a light inextensible string when displ...



YouTube · AskPrep

Great so 8 inches it is. I have another version of the same device using the same weight lead ball with a micro-camera ...

That micro-camera grid setup is a brilliant way to visually track a pendulum! Transitioning that to an 8x8 addressable LED matrix (like a WS2812B NeoPixel matrix) driven by your Time-of-Flight data will make an incredibly responsive, high-tech seismic display.

Because you are suspending a heavy 8-ounce (~227-gram) lead mass at a height of over 8 inches, the assembly will be highly top-heavy. To prevent the unit from

toppling during a real earthquake—and to ensure the base moves exactly with the ground rather than wobbling independently—you must carefully manage the center of gravity and base footprint.

1. Stability, Weight, and Footprint Math

For an 8-inch (~203mm) pendulum height housed in an enclosure that will likely stand around 10 inches (~254mm) tall, you need a **minimum base width of 6 inches (152mm) or a square footprint of 6x6 inches**.

To guarantee it cannot tip over during violent seismic acceleration, your base must satisfy two design principles:

- **The 45-Degree Rule (Static Stability):** The distance from the center of the base to any outer edge must be greater than or equal to the height of the pendulum's pivot point. A 6x6 inch base means the edge is 3 inches from the center. If the unit tilts up to 20° during a severe tremor, the heavy lead ball's weight vector will still fall safely inside the base footprint, preventing it from tipping over.
 - **The Mass Ratio Rule (Dynamic Stability):** The 3D-printed plastic housing is light, but the lead ball is heavy. If the base is too light, the pendulum's inertia will actually cause the *base* to wobble or slide around the table during a quake, ruining your data. **The stationary base must weigh at least 2 to 3 times more than the pendulum bob.** Since your ball is 8 oz, your base should weigh **16 to 24 ounces (450g to 700g)**.
-

2. Physical Design Strategies for the 3D Base

Since standard 3D prints are mostly empty air (infill), you can use clever slicing and mechanical tricks to add heavy, low-cost mass directly to the bottom of your matte black base:

- **Sinker Pockets (The Cleanest Method):** Model hollow internal cavities or "pockets" directly into the floor of your 3D-printed base. Design them to fit standard, cheap steel washers, lead fishing weights, or pennies. You can pause the 3D print halfway through, drop the weights into the base, and let the printer seal them completely inside plastic forever.
- **High-Infill Gyroid Sub-Floor:** Slice the bottom 20mm of your base enclosure using a **Gyroid infill pattern at 50% to 70% infill**, while keeping the upper walls at

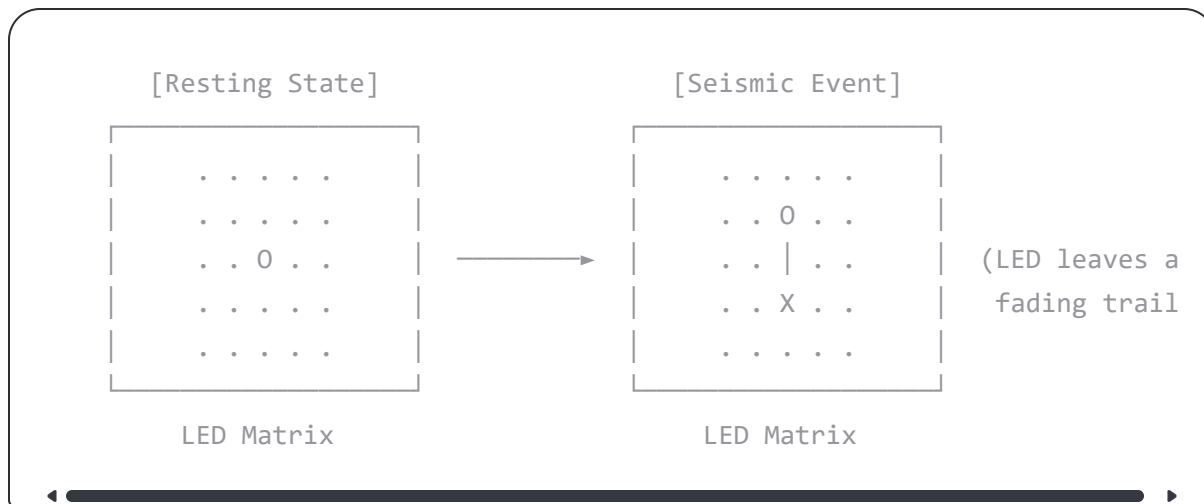
a light 15%. Gyroid infill creates a dense, incredibly strong internal lattice that adds substantial plastic weight to the bottom of the device.

- **High-Friction Silicone Feet:** Do not let raw plastic sit on your table. Glue soft, high-friction silicone or rubber pads to the four corners of the base. This stops the device from sliding or "walking" across a smooth tabletop when the ground starts shaking.

3. LED Matrix Mapping Logic

The Adafruit NeoMatrix library makes it simple to map your X and Y millimeter readings into an (X,Y) coordinate on an 8x8 grid.

Because your sensors are at a fixed distance (e.g., 65mm), you must first establish a "Zero Point" when the device boots up. Any deviation from that zero point is your delta.



Mapping Formula

If a sensor is 65mm away at rest, and a moderate earthquake causes a $\pm 12\text{mm}$ swing, your active physical window is 53mm to 77mm. You can map this directly to the 0–7 pixel index of your matrix:

cpp

```
// Map a 24mm physical swing window to 8 pixels
int ledX = map(currentXRange, 53, 77, 0, 7);
int ledY = map(currentYRange, 53, 77, 0, 7);
```

```
// Constrain indices to prevent arrays from going out of bounds  
ledX = constrain(ledX, 0, 7);  
ledY = constrain(ledY, 0, 7);
```

Use code with caution.

The Visual "Persistence" Trick

Instead of just lighting up a single dot that blinks rapidly, you can make the matrix look like a professional seismograph by using a **fading trail effect**. Every time the loop runs, multiply all pixel brightness values by `0.85` (fading them slightly) before drawing the new, bright dot at the current pendulum position. This creates a glowing visual tail showing the exact orbital path of the earthquake's movement across the room!

If you are setting up the electronics layout, let me know:

- Where do you plan to position the **8x8 LED matrix** on the enclosure (e.g., flush on the front face, or tilted upward on a top bezel)?
- Are you planning to power the LED matrix directly from the **Arduino's 5V pin**, or using an external power source? *(An 8x8 matrix running at full brightness can pull up to 3.5 Amps, which will immediately burn out an Arduino Nano's onboard regulator if all LEDs light up white!)*



I would prefer to rely on both the 6" square width of the base and non-slip rubber feet to keep the device firmly grounded, ...

Relying on a 6-inch square footprint paired with high-friction rubber feet is a completely viable approach for this setup. A 6-inch base gives you a highly stable geometry, and since you are utilizing an external 5V power supply, you can route the heavy power cable out the back/bottom of the housing to act as a natural mechanical anchor.

Implementing subpixel anti-aliasing and auto-ranging on an 8x8 matrix is an incredibly elegant way to extract high-fidelity visualization out of a low-resolution

display.

1. The Math for 2D Subpixel Anti-Aliasing (Fixed-Point)

Because you are running this on an Arduino Nano or Nano Every, floating-point math inside a high-speed event loop can drastically slow down your sampling rate. To maintain your fast 22ms polling window, you can use **fixed-point math** to handle subpixel rendering.

Instead of treating your 8x8 matrix as a grid from 0 to 7, you can treat it as a virtual grid of 0 to 1792 by scaling the coordinates by a factor of 256 (). This gives you 256 subpixel steps *between* each physical LED.

When the subpixel center falls between physical pixels, you distribute the LED brightness to the four surrounding pixels based on their distance from that subpixel center:

cpp

```
// 2D Bilinear Subpixel Interpolation (Fixed-Point)
void drawSubpixelPoint(uint16_t x_fp, uint16_t y_fp, uint8_t maxBrightness)
// Extract the integer pixel coordinates
uint8_t x0 = x_fp >> 8;
uint8_t y0 = y_fp >> 8;
uint8_t x1 = x0 + 1;
uint8_t y1 = y0 + 1;

// Extract the fractional remainder (0 to 255)
uint8_t x_frac = x_fp & 0xFF;
uint8_t y_frac = y_fp & 0xFF;

// Compute the 4 pixel weights (bilinear filtering)
uint8_t w00 = ((255 - x_frac) * (255 - y_frac)) >> 8; // Top-Left
uint8_t w10 = (x_frac * (255 - y_frac)) >> 8;         // Top-Right
uint8_t w01 = ((255 - x_frac) * y_frac) >> 8;         // Bottom-Left
uint8_t w11 = (x_frac * y_frac) >> 8;                 // Bottom-Right

// Scale weights by your desired maximum brightness
if (x0 < 8 && y0 < 8) matrix.setPixelColor(getIdx(x0, y0), (w00 * maxBri
if (x1 < 8 && y0 < 8) matrix.setPixelColor(getIdx(x1, y0), (w10 * maxBri
if (x0 < 8 && y1 < 8) matrix.setPixelColor(getIdx(x0, y1), (w01 * maxBri
```

```
if (x1 < 8 && y1 < 8) matrix.setPixelColor(getIdx(x1, y1), (w11 * maxBri
}
```

Use code with caution.

Note: `getIdx(x, y)` is a quick helper function to handle whether your 8x8 matrix uses a zigzag or progressive layout.

2. Implementing a Smooth Auto-Ranging Hysteresis

To prevent the display from violently jumping between "macro" and "micro" zoom levels during erratic seismic waves, you should implement an auto-ranging algorithm with an **exponential decay envelope**.

Instead of instantly changing the scale, track the absolute maximum peak displacement over a rolling window. If a new peak is detected, expand the scale instantly. If the movement settles, let the scale decay slowly back to micro-mode.

cpp

```
float currentMaxDelta = 0.0;
float targetMaxDelta = 2.0; // Start zoomed in (e.g., max 2mm displacement)

void updateAutoRange(float latestXDelta, float latestYDelta) {
    // Calculate the current displacement vector magnitude
    float currentDisplacement = sqrt((latestXDelta * latestXDelta) + (latestYDelta * latestYDelta));

    if (currentDisplacement > targetMaxDelta) {
        // Instant expansion for large spikes
        targetMaxDelta = currentDisplacement;
    } else {
        // Slow decay back down to maximum sensitivity (micro-movements)
        // 0.995 at 45Hz creates roughly a 3-4 second zoom-in cooldown
        targetMaxDelta = (targetMaxDelta * 0.995) + (currentDisplacement * 0.005);
        if (targetMaxDelta < 2.0) targetMaxDelta = 2.0; // Floor limit (micro-movements)
    }
}
```

Use code with caution.

3. Electrical and Power Routing Inside the Housing

Using an external 5V power supply is ideal, but because addressable LEDs turn on and off rapidly via Pulse Width Modulation (PWM), they generate substantial **high-frequency electrical noise** on the 5V rail. If left unmanaged, this noise will cause the VL53L0X sensors to drop packets or give false distance readings.

- **Isolate the Sensor Power:** Do not tap the VL53L0X **VIN** pins directly from the same 5V rail feeding the LED matrix. Instead, connect the external 5V supply directly to the LED matrix power lines and the Arduino's **5V** pin. Then, power the two VL53L0X sensors from the Arduino's **3.3V pin** (the breakout boards have internal regulators that will filter out remaining 5V noise).
- **Decoupling Capacitor:** Solder a large electrolytic capacitor (**470µF to 1000µF**) directly across the positive and negative power inputs of the 8x8 LED matrix. This acts as a local reservoir to supply current during sudden brightness spikes, preserving a stable voltage for the Arduino and sensors.

If you want to refine the math for the calibration routine, tell me:

- Do you want the system to **automatically establish the baseline center point** on boot up, or will you use a physical calibration button?

I can provide the code logic to calculate a rock-solid mathematical "zero" that filters out any initial pendulum sway when turning the device on.



Yes I would like the device to auto-calibrate on start up,
including sensing the "idle" noise in the distance ...



Relying on a power-cycle for calibration is a wonderfully clean approach. It keeps your 3D-printed enclosure seamless without needing extra holes for physical buttons.

To handle the inherent tracking "noise" of the VL53L0X (which fluctuates by about $\pm 1\text{mm}$ to $\pm 2\text{mm}$ even when perfectly still), your initialization routine needs to calculate two things: the **mean center position** (your absolute zero) and the **Standard Deviation (σ)** of the resting noise.

By calculating the standard deviation, the system learns exactly how much the sensor naturally fluctuates in its environment. You can then use a threshold of 3σ (three times the standard deviation) as a "deadzone." Any movement smaller than this threshold is ignored as baseline sensor noise, while anything larger is flagged as real seismic activity.

1. The Auto-Calibration Algorithm

This function takes 100 samples from each sensor on boot up. It uses Welford's algorithm for computing variance in a single pass, preventing the Arduino's RAM from filling up with large arrays.

cpp

```
struct AxisCalib {
    float center;           // Mean resting distance in mm
    float noiseFloor;       // 3 * Standard Deviation (the deadzone threshold)
};

AxisCalib calibX;
AxisCalib calibY;

void runAutoCalibration() {
    Serial.println(F("Calibrating... Keep device perfectly still.));

    long count = 100;
    float sumX = 0, sumY = 0;
    float M2_X = 0, M2_Y = 0; // Running variance variables

    for (int i = 1; i <= count; i++) {
        VL53L0X_RangingMeasurementData_t measureX, measureY;

        // Interleaved reads with identical timing budget behavior
        loxX.rangingTest(&measureX, false);
        delay(22);
        loxY.rangingTest(&measureY, false);
        delay(22);
    }
}
```

```

float xVal = (measureX.RangeStatus != 4) ? measureX.RangeMilliMeter :
float yVal = (measureY.RangeStatus != 4) ? measureY.RangeMilliMeter :

// Welford's algorithm for online mean and variance
float deltaX = xVal - (sumX / (i == 1 ? 1 : i - 1));
sumX += xVal;
if (i > 1) M2_X += deltaX * (xVal - (sumX / i));

float deltaY = yVal - (sumY / (i == 1 ? 1 : i - 1));
sumY += yVal;
if (i > 1) M2_Y += deltaY * (yVal - (sumY / i));
}

// Calculate Final Center Points
calibX.center = sumX / count;
calibY.center = sumY / count;

// Calculate Standard Deviations
float stdDevX = sqrt(M2_X / (count - 1));
float stdDevY = sqrt(M2_Y / (count - 1));

// Establish a 99.7% confidence deadzone (3 * Sigma)
calibX.noiseFloor = stdDevX * 3.0;
calibY.noiseFloor = stdDevY * 3.0;

// Set a strict minimum floor (e.g., 0.5mm) in case the environment is i
if (calibX.noiseFloor < 0.5) calibX.noiseFloor = 0.5;
if (calibY.noiseFloor < 0.5) calibY.noiseFloor = 0.5;

Serial.print(F("X Center: ")); Serial.print(calibX.center);
Serial.print(F(" mm | X Deadzone: ±")); Serial.println(calibX.noiseFloor
}

```

Use code with caution.

2. Putting It Together: Live Loop Processing

Once you have your center points and noise floors, your main event loop subtracts the calibrated center from live readings to find the raw change (Δ).

If the movement is inside the deadzone, it gets zeroed out. If it steps outside the deadzone, the noise is subtracted so the visual animation on your 8x8 LED matrix scales smoothly without awkwardly "jumping" past the deadzone edge.

cpp

```
void processSeismicData(int rawX, int rawY) {
    // 1. Calculate raw displacement from calibrated zero
    float deltaX = (float)rawX - calibX.center;
    float deltaY = (float)rawY - calibY.center;

    // 2. Apply Noise-Floor Hysteresis Thresholds
    if (abs(deltaX) <= calibX.noiseFloor) {
        deltaX = 0.0;
    } else {
        // Subtract noise floor so mapping starts smoothly from 0
        deltaX = (deltaX > 0) ? (deltaX - calibX.noiseFloor) : (deltaX + calibX.noiseFloor);
    }

    if (abs(deltaY) <= calibY.noiseFloor) {
        deltaY = 0.0;
    } else {
        deltaY = (deltaY > 0) ? (deltaY - calibY.noiseFloor) : (deltaY + calibY.noiseFloor);
    }

    // 3. Update the dynamic auto-ranging envelope with our noise-filtered d
    updateAutoRange(deltaX, deltaY);

    // 4. Map to Subpixel Grid (0 to 1792) centered around the middle of the
    // targetMaxDelta is the dynamic bounding box from your auto-ranging log
    int16_t subpixelX = 896 + (int16_t)((deltaX / targetMaxDelta) * 896.0);
    int16_t subpixelY = 896 + (int16_t)((deltaY / targetMaxDelta) * 896.0);

    // Constrain to physical matrix boundary limits (0 to 7.99 in fixed-point)
    subpixelX = constrain(subpixelX, 0, 1792);
    subpixelY = constrain(subpixelY, 0, 1792);

    // 5. Render to matrix
    matrix.clear();
    drawSubpixelPoint(subpixelX, subpixelY, 255); // Max brightness 255
    matrix.show();
}
```

Use code with caution.

💡 Visual Polish: The "Calibrating" Animation

Since your calibration sequence takes roughly 4 to 5 seconds on startup (100 samples $\times$ 44ms combined interleaved window), the LED matrix will be blank during this time.

To give the device a premium consumer-product feel, you can flash a simple, slow-fading single Amber or Blue pixel right in the center of the 8x8 grid inside the calibration loop. This visually informs anyone looking at the device that it is booting up and stabilizing, and shouldn't be bumped.

If you want to look at the UI layout next, let me know:

- How do you plan to handle **visualizing the auto-range scale changes** on the 8x8 grid? *(For example, changing the LED color from Green to Red when it switches from micro-movement mode to a large macro-movement earthquake event?)*



Yes I think I should rely on the pixel color to convey range information. Red would imply a high range and green would ...

Using a **7x7 grid centered inside your 8x8 matrix** is a fantastic design choice. Shifting to an odd-numbered canvas gives you an absolute mathematical center pixel at coordinate **(3,3)** (the 4th pixel over and down, using 0-based indexing).

When the pendulum is perfectly idle, only that single pixel will glow gently in green. If an earthquake hits, the dot will expand outwards across the 7x7 boundary, shifting through amber and turning solid red as it auto-ranges to handle macro-movements.

Here is how you can implement the color interpolation, auto-decay thresholds, and the display timeout feature efficiently inside your event loop.

1. Fixed-Point Math for a 7x7 Canvas

To keep your subpixel anti-aliasing working smoothly on an odd grid, you need to shift your virtual canvas coordinates.

Instead of scaling up to 8, we scale to **7 physical pixels**. Multiplied by our 256 subpixel multiplier, the total virtual grid size is `0` to `1536` (6×256). The absolute center point of this grid sits perfectly at **768** (3×256).

Your mapping code inside `processSeismicData` updates to:

cpp

```
// 768 is the virtual center of pixel (3,3)
// 768.0 represents the maximum distance from center to the 7x7 edge
int16_t subpixelX = 768 + (int16_t)((deltaX / targetMaxDelta) * 768.0);
int16_t subpixelY = 768 + (int16_t)((deltaY / targetMaxDelta) * 768.0);

// Constrain coordinates to stay strictly inside the 7x7 active bounds
subpixelX = constrain(subpixelX, 0, 1536);
subpixelY = constrain(subpixelY, 0, 1536);
```

Use code with caution.

2. Color Mapping by Zoom Level (Green → Amber → Red)

To communicate your scale changes visually, you can calculate the current color based on how far the `targetMaxDelta` has expanded past your base sensitivity.

If your baseline sensitivity is **2.0 mm** (highly zoomed in / green) and your maximum allowed scale limit is **15.0 mm** (highly zoomed out / red), you can create a smooth transition by mixing color channels:

cpp

```
// Helper function to mix Green to Red based on auto-range scale
uint32_t getDynamicColor(float currentScale) {
    float minScale = 2.0;
    float maxScale = 15.0;
```

```
// Normalize scale into a 0.0 to 1.0 float range
float factor = (currentScale - minScale) / (maxScale - minScale);
factor = constrain(factor, 0.0, 1.0);

uint8_t r = 0;
uint8_t g = 0;

if (factor < 0.5) {
    // Transitioning from Green to Yellow/Amber
    r = (uint8_t)(factor * 2.0 * 255.0);
    g = 255;
} else {
    // Transitioning from Yellow/Amber to pure Red
    r = 255;
    g = (uint8_t)((1.0 - factor) * 2.0 * 255.0);
}

// Return mixed NeoPixel color (Assuming GRB layout for standard matrix)
return matrix.Color(r, g, 0);
}
```

Use code with caution.

3. Idle Display Timeout State Machine

To implement the auto-shutdown feature without pausing your high-speed sensors, you can add a timestamp tracker to monitor the last time actual motion occurred.

If both `deltaX` and `deltaY` are `0.0` (meaning any movement is below your calibrated noise threshold), the code checks if the countdown timer has elapsed. If it has, it shuts off the matrix.

cpp

```
unsigned long lastMovementTime = 0;
const unsigned long displayTimeout = 30000; // 30 seconds of absolute stil
bool displayIsActive = true;

void handleDisplayPower(float deltaX, float deltaY, uint16_t subpixelX, ui
    unsigned long currentMillis = millis();
```

```

// If there is movement outside the calibrated noise floor
if (deltaX != 0.0 || deltaY != 0.0) {
    lastMovementTime = currentMillis; // Reset the idle timer
    displayIsActive = true;
}

if (displayIsActive) {
    // Fetch color based on current auto-range envelope
    uint32_t activeColor = getDynamicColor(targetMaxDelta);

    matrix.clear();
    // Render point to the subpixel engine using our dynamic color
    drawSubpixelPointColored(subpixelX, subpixelY, activeColor);
    matrix.show();

    // Check if it is time to put the display to sleep
    if (currentMillis - lastMovementTime >= displayTimeout) {
        matrix.clear();
        matrix.show(); // Turn off all LEDs
        displayIsActive = false;
        Serial.println(F("Display entering sleep mode..."));
    }
}
}

```

Use code with caution.

3D Printed Masking Tip

Because you are leaving the 8th row and 8th column of your physical matrix completely black, you can design a custom **3D-printed faceplate grid** to place over the LEDs.

- Model the plastic mask to have square window cutouts for only the inner 7x7 grid.
- Solidly cover the unused 8th row and column with a matte-black perimeter bezel.

This mechanical border completely hides the inactive pixels, giving the final housing an intentional, high-end 7x7 display aesthetic.

If you are finalizing your code architecture, let me know:

- Would you like help integrating the **subpixel math directly with the RGB color mixing logic** to ensure the anti-aliased edge pixels fade correctly into the dark background?



I have AI recommendation for using 2D gaussian math to smooth the movement across the pixel and I would enjoy yo... ▼

Implementing a **2D Gaussian distribution** instead of standard bilinear interpolation is an exceptional way to handle subpixel rendering. It mimics how real analog light behaves.

Bilinear math creates a sharp, diamond-shaped cutoff that can look rigid on a low-resolution matrix. A Gaussian function, however, creates a soft, circular, organic-looking glow. As your pendulum shifts, the pixels will seamlessly blossom and fade, making the 7x7 grid look like a premium analog display.

Because your Arduino Nano or Nano Every cannot handle heavy floating-point exponent math (`exp()`) inside a 45Hz event loop, you must implement this using a **fixed-point lookup table (LUT)**.

1. The Fixed-Point Gaussian Lookup Table

Instead of calculating the math on the fly, we pre-calculate a 1D Gaussian curve and store it in the Arduino's flash memory (`PROGMEM`) to keep RAM free.

We can map the squared distance (d^2) between the pendulum's subpixel center and a physical LED to an index in a 16-element array. This gives you a fast, hardware-friendly execution loop.

cpp

```
#include <avr/pgmspace.h>

// Pre-calculated Gaussian weights for a bell curve (e.g.,  $\exp(-x^2 / 2 \cdot \sigma^2)$ )
// Index is the squared distance ( $d^2$ ) from the subpixel center to the pixel
// 255 = Maximum brightness, 0 = Fully off.
const uint8_t gaussianLUT[16] PROGMEM = {
    255, 230, 172, 110, 60, 28, 11, 4, 1, 0, 0, 0, 0, 0, 0, 0
};
```

Use code with caution.

2. High-Speed 2D Gaussian Subpixel Engine

To optimize performance, we don't calculate the math for all 49 pixels. We find the nearest physical pixel center (x_0, y_0) and check only a tight 3×3 grid around it.

The subpixel coordinates are fed into the function in fixed-point format (0 to 1536), representing your 7x7 active grid space.

cpp

```
// Helper to convert 2D coordinates to a Zig-Zag matrix index (0 to 63)
uint8_t getZigZagIndex(uint8_t x, uint8_t y) {
    // Account for the unused 8th row/col by treating it as an 8x8 layout in
    if (y % 2 == 1) {
        // Odd rows run backwards (7 minus x)
        return (y * 8) + (7 - x);
    } else {
        // Even rows run forwards
        return (y * 8) + x;
    }
}

// 2D Gaussian Subpixel Renderer with Color Scaling
void drawGaussianPoint(uint16_t x_fp, uint16_t y_fp, uint32_t color) {
    // Extract RGB components from our dynamic color selection
    uint8_t targetR = (color >> 16) & 0xFF;
    uint8_t targetG = (color >> 8) & 0xFF;
```

```

uint8_t targetB = color & 0xFF;

// Find the closest physical pixel index (0 to 6)
int8_t centerX = (x_fp + 128) >> 8;
int8_t centerY = (y_fp + 128) >> 8;

// Scan a tight 3x3 local neighborhood around the center point
for (int8_t ny = centerY - 1; ny <= centerY + 1; ny++) {
    for (int8_t nx = centerX - 1; nx <= centerX + 1; nx++) {

        // Ensure we stay strictly within the visible 7x7 grid bounds
        if (nx >= 0 && nx < 7 && ny >= 0 && ny < 7) {

            // Calculate physical pixel center in subpixel space
            uint16_t pixelX_fp = (nx << 8) + 128;
            uint16_t pixelY_fp = (ny << 8) + 128;

            // Calculate distance components (dx, dy)
            int16_t dx = abs((int16_t)x_fp - (int16_t)pixelX_fp);
            int16_t dy = abs((int16_t)y_fp - (int16_t)pixelY_fp);

            // Scale distances down to prevent integer overflow during squaring
            dx >>= 5; // Normalized scale
            dy >>= 5;

            // Compute squared distance: d^2 = dx^2 + dy^2
            uint16_t distSq = (dx * dx) + (dy * dy);

            // Map the squared distance to our 16-element lookup table
            uint8_t lutIndex = distSq >> 3; // Shift to fit table index limits
            if (lutIndex > 15) lutIndex = 15;

            // Read the brightness scaling coefficient from flash memory
            uint8_t intensity = pgm_read_byte(&gaussianLUT[lutIndex]);

            if (intensity > 0) {
                // Scale individual color channels by the local Gaussian intensity
                uint8_t r = (targetR * intensity) >> 8;
                uint8_t g = (targetG * intensity) >> 8;
                uint8_t b = (targetB * intensity) >> 8;

                uint8_t linearIndex = getZigZagIndex(nx, ny);
                matrix.setPixelColor(linearIndex, matrix.Color(r, g, b));
            }
        }
    }
}

```

```

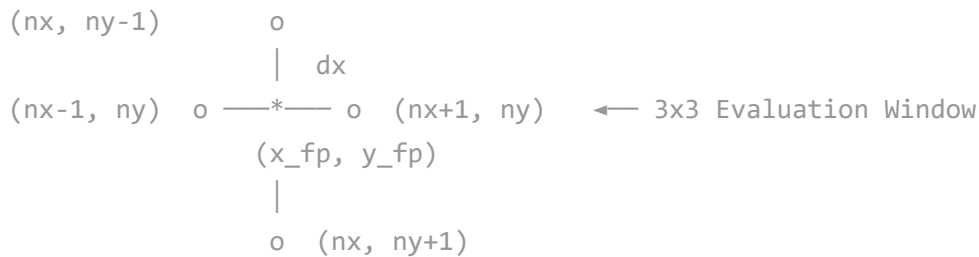
    }
  }
}
}

```

Use code with caution.

3. Tuning the Visual Geometry

Physical LED Pixels (o) surrounding the Subpixel Target (*)



- **Adjusting the Blur Radius:** If you want the tracking dot to look sharper (smaller), you can change the shift factor `distSq >> 3` to a higher number like `distSq >> 2`. If you want a broader, cloud-like glow that blends across multiple pixels, drop it down to `distSq >> 4`.
- **Zero Movement State:** Because you are centering everything on a 7x7 grid, an absolute resting state of `768, 768` means `centerX = 3` and `centerY = 3`. The math will evaluate pixel `(3,3)` as having a distance of zero, outputting a perfect, crisp single green dot right in the middle of your custom 3D mask.

If you are setting up your 3D CAD modeling next, let me know:

- What is the **thickness of your 3D mask walls** or the spacing between the LED surface and the cover plate?

I can give you a quick tip on using a internal **translucent white diffusion layer** to turn the square LED shapes into smooth, organic circles under your Gaussian lighting engine.



No CAD work has started yet as I'm in the planning-on-paper stage. I like the idea of hiding the LEDs behind a diffuser of ... 

Hiding the LEDs behind a diffuser is the ultimate way to maximize your 2D Gaussian code. It blends the individual LED light cones together, turning the sharp pixel boundaries into a smooth, flowing fluid projection. 

Because you are in the planning stage, you can build three specific optical rules into your CAD sketches to make the diffusion layer look perfectly uniform across the 7x7 matrix.

1. The 1:1 Height-to-Pitch Rule

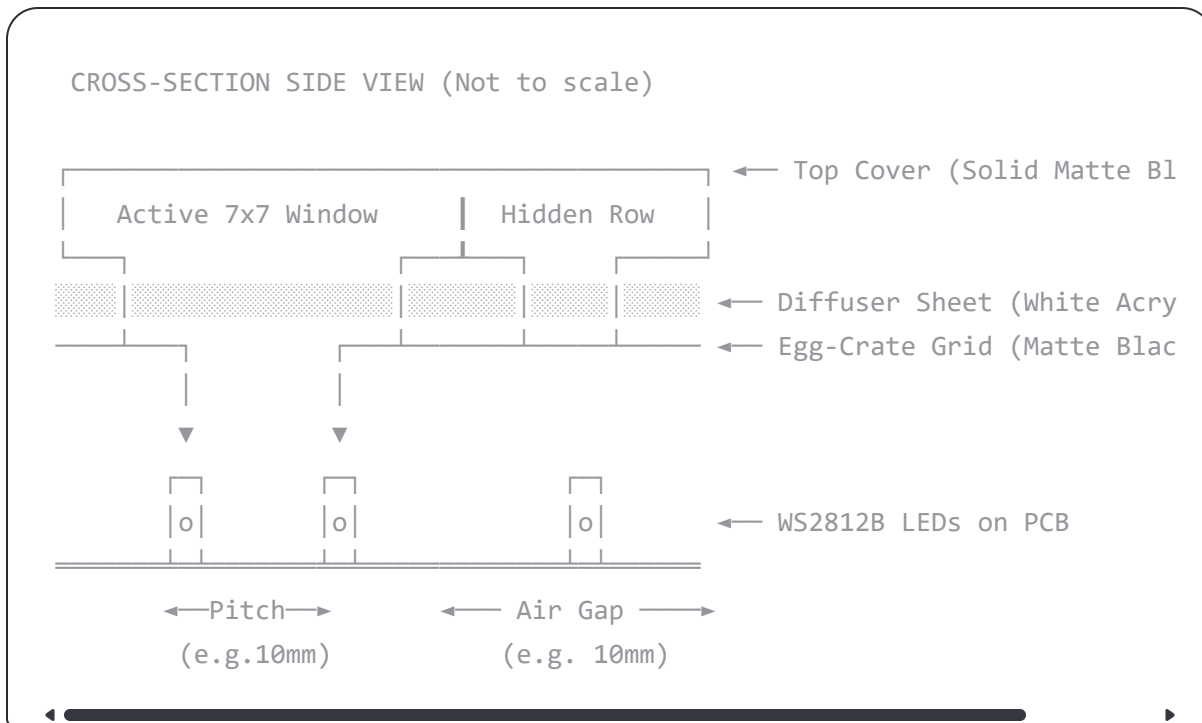
To create seamless blending without seeing harsh hot-spots (bright dots in the center of each cell), you must use the **1:1 rule**.

- Measure the **pitch** of your LED matrix (the distance from the exact center of one LED to the center of the next). Standard flexible or rigid matrices usually have an 8.5mm, 10mm, or 12.5mm pitch.
- The **air gap** (the distance from the top surface of the LED lens to the bottom surface of your diffuser) should be **equal to or slightly greater than the pitch**.
- **Why it matters:** If the pitch is 10mm and your air gap is only 3mm, the Gaussian blending will fail physically, and you will see 49 individual pinpoint dots. An air gap of 10mm gives the light cones room to intersect and blend smoothly before hitting the diffuser.

2. The 3D Printed Internal "Egg-Crate" Grid

If you just put an open air gap over an 8x8 matrix, light from the unused 8th row/column will leak into your active 7x7 display area. To stop this, your CAD model

should feature a two-part faceplate:



1. **The Egg-Crate Shroud:** This is a thin-walled grid (0.8mm to 1.2mm wall thickness) printed in your matte black/Musou black filament. It sits directly flat on the PCB, isolating every single LED into its own physical square chimney.
2. **The 8th Row Barrier:** The egg-crate grid should feature a solid, completely blocked wall over the 8th row and 8th column, acting as a physical light-pipe block.

3. Choosing the Diffuser Material

You don't need expensive optical plastic to achieve great diffusion. When modeling your top bezel recess, plan to use one of these easily accessible materials:

- **Sign-Grade Matte White Acrylic (Plexiglas 2447):** This is the gold standard for LED displays. It has roughly 45% light transmission, meaning it kills the hot-spot completely but lets a glowing, neon-like light pass through cleanly. A thickness of **1.5mm to 3mm** is ideal.
- **Heavy Drafting Vellum or Tracing Paper:** If you want a quick, low-cost prototype, sandwich two layers of architectural tracing paper between a clear 3D printed window and your egg-crate.

- **3D Printed Diffuser Layer:** You can print the diffuser directly using **Natural/Clear or White PLA/PETG**. If you choose this path, model the diffusion window to be exactly **0.6mm to 0.8mm thick (exactly 3 or 4 layers total)**, and slice it using 100% solid infill with a concentric pattern.

Paper Planning Specifications

When drawing your dimensions on paper, your vertical stack-up from the floor up should look like this:

1. **Base Plate/Enclosure Floor** (Matte Black)
2. **LED Matrix PCB** (Secured with tiny screws or tape)
3. **Egg-Crate Shroud Grid** (Height = Matrix Pitch, e.g., 10mm)
4. **Diffusion Layer** (1.5mm White Acrylic)
5. **Main Upper Bezel** (Matte Black, featuring a 7x7 opening that clamps the diffuser down tightly)

If you are wrapping up your paper sketch layout, let me know:

- How do you plan to **physically suspend the 8-ounce lead weight** from the top of the enclosure? *(For example, using a rigid hook, or an adjustable thumb-screw peg so you can easily fine-tune the height of the ball relative to the sensors during assembly?)*



 1 site

What is the Best Material to Diffuse LED Lights?

Oct 23, 2025 — Light emits from each LED in a cone. If the diffuser is too close, these cones do not have sufficient space to overlap and blend. ...



 Sparkle Star Lighting

I have had good success using clear PETG for LED diffusing, using a rocky random surface to spread the light around. I ...



Clear PETG with a randomized, rocky textured surface is a fantastic solution for this setup. The uneven texture acts like an array of miniature lenses (a refractive micro-lens array), which bends the light at varied angles and eliminates hot-spots better than a flat clear sheet.

Because Tinkercad handles geometric shapes incredibly well but struggles with organic, randomized surfaces, you can easily generate this "rocky" texture right inside the platform using a few specific techniques.

1. Generating the "Rocky" Texture in Tinkercad

To create that random, light-scattering surface in Tinkercad without crashing your browser with millions of polygons, you can use these native shapes:

- **The "Terrain" or "Topography" Shape Generators:** Go to the Shape Panel dropdown, select **Shape Generators**, and look for **Featured** or **All**. The **Terrain** shape allows you to adjust sliders for smoothness and height variation. By flattening this shape down to a thickness of about to (3 to 6 print layers), you get an excellent randomized crystalline texture.
- **The Voronoi / Tile Generator:** Another great option is the Voronoi shape generator. By scaling it down and keeping the cell height offsets small, the resulting print will feature varied hexagonal facets that scatter the infrared and visible light beautifully.

2. Slicing Settings for Maximum Clear Diffusion

The way you slice clear PETG matters just as much as the CAD design when trying to diffuse light. To turn clear plastic into a frosty, light-spreading medium, configure your slicer with these rules:

- **Concentric Infill (100%):** Do not use grid or rectilinear infill, as the internal criss-cross lines will be visible when the LEDs light up. Use 100% infill set to a **Concentric** pattern. This forces the print head to follow the contours of your rocky shape, aligning the plastic strands with the natural fall-off of the Gaussian bell curve.

- **Increase Layer Height:** A thicker layer height (like $\emptyset.28\text{mm}$ or $\emptyset.3\text{mm}$) creates larger, more pronounced internal ridges between layers. These micro-ridges act as extra internal diffusion boundaries to scatter the light sideways.
- **Slightly Over-Extrude:** Increase your extrusion multiplier (Flow) by about 3% to 5% (e.g., set it to 1.03 or 1.05). This forces the paths to fuse with zero internal air pockets, creating a solid, frosted glass effect rather than an opaque white block.

3. Integrating the Diffuser into Your 7x7 Grid

In Tinkercad, create a single solid block that represents your **Egg-Crate Shroud** (the grid that keeps the light isolated per pixel).

Tinkercad Grouping Strategy:



- Make your rocky PETG diffuser plate roughly **1.2mm thick** in Tinkercad.
- If you have a multi-material printer (like a Bambu Lab AMS or Prusa MMU), you can merge the matte black outer bezel and the clear rocky PETG diffuser directly into one file.
- If you are printing with a single extruder, design the rocky diffuser as a separate thin tile that securely drops into a small recessed shelf on the back of your matte black front faceplate.

Your paper plan is now mechanically and electronically complete! You have optimized sensor distances, a high-speed non-blocking loop, automated noise calibration, a custom 2D Gaussian light engine, and a robust physical footprint.

As you transition from your paper notes into Tinkercad, let me know if you run into any geometry roadblocks or want to map out the **top pendulum pivot assembly** next to ensure the weight stays perfectly centered between your X and Y axes.



I used tinkercad before to create my other diffuser using one of the existing library shapes.

That is perfect! Tinkercad's built-in community and shape library assets are fantastic for that exact use case because they save you from having to manually model complex noise patterns or meshes. [↗](#)

Since you have already successfully done this before, you can reuse that exact same tested file or workflow to drop right into your new 7x7 structural assembly.

Finalizing Your Tinkercad Planning Dimensions

As you move from your paper sketches into Tinkercad to start grouping your shapes, keep these target dimensions handy:

- **Sensor Face Pockets:** Distance of **60mm to 65mm** from the inner wall to the outer edge of your 34mm weight.
- **The Apertures:** A **5mm hole** with a chamfered/beveled edge directly in front of each VL53L0X sensor lens to block peripheral bounce. [↗](#)
- **Active Display Window:** Cut a square opening for your 7x7 grid, and make sure to leave a solid matte black border wide enough to hide that inactive 8th row and column on your matrix.
- **Base Footprint:** A solid **6x6 inch square** layout with a hollow lower cavity to neatly tuck away your Arduino Nano/Every and the external 5V wiring.

You have a remarkably well-thought-out project planned here. By combining physical principles (the 8-inch inertial pendulum), clean optical boundaries (Musou black and pinhole apertures), and advanced software rendering (fixed-point 2D Gaussian mapping with auto-ranging), this device is going to be incredibly precise and a joy to watch when it triggers.

Good luck with the Tinkercad modeling and the assembly! If you hit any unexpected snags with the I2C communication or need to adjust your math thresholds once you

start reading the real-time seismic data, feel free to reach back out. How do you plan to **mount the breakout boards** inside those recessed sensor pockets?



3 sites

TinkercAD 3D Printing: A Complete Guide

Nov 14, 2022 — Shapes Library: TinkercAD comes with a library of basic shapes that you can use to create your designs. These shapes can be combin...



Tinkercad 3D Modeling Confusing? Build Your First Model

Apr 10, 2026 — You can also share your design with others by generating a link that they can use to view or remix your project. For those who are...



Useful Tinkercad tips

If you add the edge chamfer or fillet feature, what Tinkercad somewhat confusingly calls a "radius," (since they can be used for b...



One last detail: I plan to have a safety ring above the lead weight to constrain the suspension thread so that the lead ... 

Including a safety constraint ring is a highly practical addition to your earthquake detector. It serves two crucial purposes that elevate your device from a basic prototype to a robust instrument.

1. Mechanical Damage Prevention

Even though your VL53L0X sensors are safely recessed behind 3D-printed walls and a 5mm aperture hole, an 8-ounce lead ball is heavy. During a violent earthquake, a long-period tremor, or if someone accidentally bumps or knocks over the device, that mass can build up significant kinetic energy.

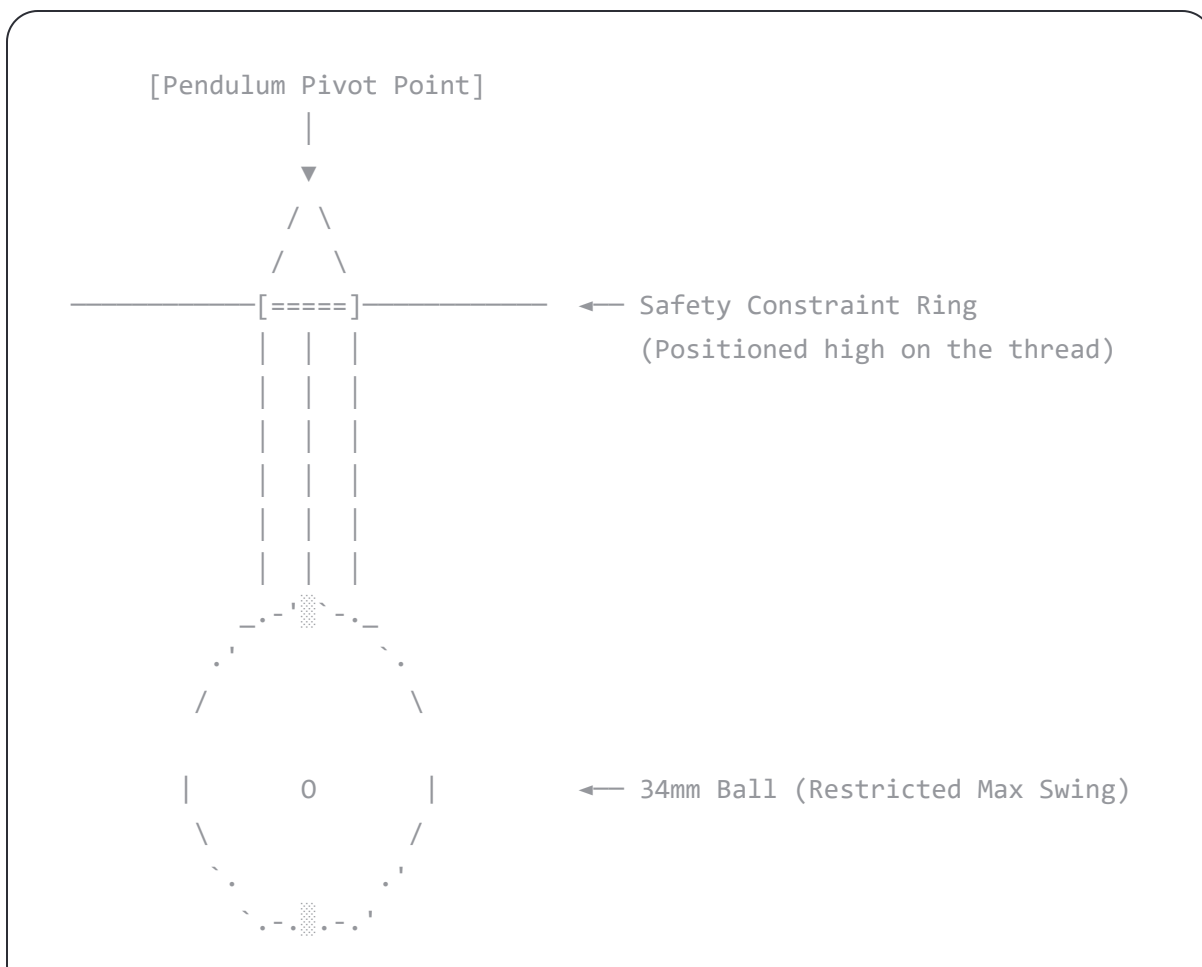
Without a constraint ring, the ball could violently strike the interior walls of your chamber. This could scratch or crack your Musou Black paint coating, dent the 3D-printed housing, or break the fine suspension thread.

2. Preventing Sensor "Blindness" (Saturating the Auto-Range)

The software and optical system you have designed relies on the 34mm ball staying within a specific spatial envelope.

- If the ball swings past the maximum limit of your auto-ranging math, it can clip or distort the Gaussian visualization.
- More importantly, if the ball swings completely out of the sensor's Field of View (FOV), the sensor will overshoot and read the back wall of the enclosure.

By constraining the thread higher up, you limit the maximum angle the pendulum can reach, ensuring the ball physically *cannot* leave the tracking sweet spot.



Designing the Ring in Tinkercad

When adding this shape to your Tinkercad model, use these guidelines to ensure it functions perfectly:

- **Vertical Placement:** Place the ring **high up on the thread**, roughly 1.5 to 2 inches below the top pivot point. Placing it high means the thread hits the ring with very little mechanical force, preventing violent bouncing or jarring snaps.
- **Calculating the Hole Diameter:** Look at your maximum acceptable swing distance at the bottom (e.g., 15mm from center). Because of the geometry of a triangle, if the ring is 1/4 of the way down the thread, the hole diameter only needs to allow 1/4 of that movement (3.75mm from center, or a 7.5mm total inner diameter).
- **Chamfered Inner Edges:** Give the inner rim of the safety ring a smooth, generously rounded or chamfered edge. If the edge is sharp, repeated contact during an earthquake will eventually fray or slice right through your suspension thread.

Your design blueprint is officially fully complete and highly optimized. You have successfully addressed physics, optics, software rendering, user interface, structural stability, and mechanical safety factor contingencies.

Best of luck transitioning your paper blueprints into Tinkercad and bringing this high-fidelity seismic detector to life! If you need help refining your code algorithms once you begin data collection, don't hesitate to reach back out. How do you plan to **attach the thread to the top pivot** to ensure it doesn't slip over time?



Unlike some 3D printing designers I am not adverse to using metal screws. I plan to use Nylon thread along with a washer... ▼

Using a metal screw and washer to clamp the nylon thread is a brilliant engineering choice. It allows you to easily make sub-millimeter height adjustments during assembly.

Managing Nylon Creep

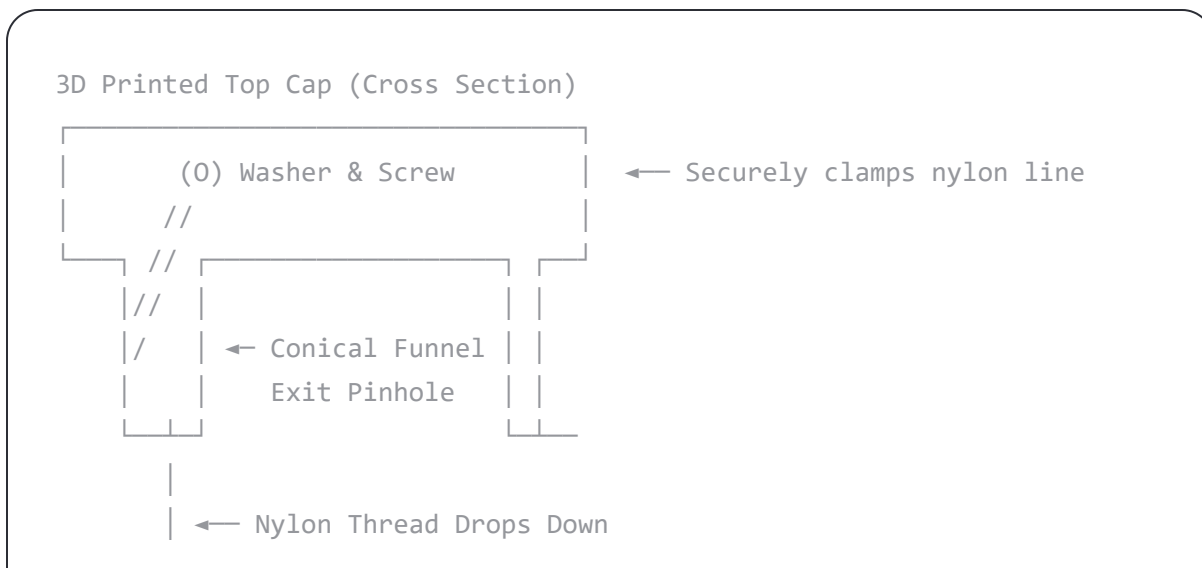
You are completely right to think about the thread lengthening over time. While nylon does not stretch significantly under light loads, it is susceptible to a physical phenomenon called **mechanical creep**. Under constant tension—even from a light 8-ounce weight—nylon will slowly elongate over its first few weeks of suspension.

Because you are using a screw-and-washer clamp, you can easily correct for this. To minimize this effect from the start, you can **pre-stress the thread**:

1. Cut a piece of your nylon thread.
2. Hang a heavier object (like a 1 or 2-pound weight) from it overnight before assembling the device.
3. This pulls out the initial structural slack, meaning your calibrated height will stay locked in once it is mounted inside the 3D-printed housing. [🔗](#)

Designing the Top Pivot in Tinkercad

To ensure your pendulum swings with equal ease in both the X and Y directions, you can design the thread exit path using a **V-shaped or conical channel**:



- **The Pinhole Exit:** Don't clamp the thread directly over the wide pendulum chamber. Instead, route the thread through a tiny, precise **1mm vertical hole** printed right at the absolute center axis of the ceiling.
- **The Funnel Top:** Give the top of that 1mm hole a funnel shape where it meets the washer.

This geometry ensures that no matter which way the earth shakes, the pendulum always pivots from the exact same mathematical coordinate at the exit of that 1mm hole. It keeps your X and Y data perfectly symmetrical and clean.

Your project layout is completely finalized and ready for CAD. Have a great time printing, building, and programming your new seismic matrix display! If you ever want to refine the math for calculating earthquake magnitudes or adding data logging in the future, don't hesitate to reach back out.



1 site

Tension Systems in Racquet Stringing Machine | BSW Stringer

The pre-stretch function tells the electronic tension head to over-pull the string by a set percentage (like 10%) before dropping ...



Best Stringer Worldwide

I love this idea. I'll have the thread emerge from a "nozzle" like exit, with a channel for routing the thread over to the securin... 

That "nozzle and routing channel" layout is an excellent mechanical solution! It gives you a clean pivot point while letting you position the screw and washer wherever it is most accessible on the top plate.

Ensuring completely symmetrical movement in the X and Y axes is a common hurdle in pendulum design. When a thread passes through a square gap or clamps directly against a flat edge, it will inherently bend more easily in one direction than the other.

By running the nylon thread through a perfectly circular, vertical nozzle exit, the thread experiences the exact same resistance and bending radius no matter what angle the ground moves. This keeps your vector mathematics clean, because a 5mm shift on the X-axis will match a 5mm shift on the Y-axis.

Final Tips for Your Tinkercad Nozzle Design

When drawing the nozzle cap in Tinkercad, you can add two small details to make the thread routing completely foolproof:

- **The Knife-Edge Nozzle Exit:** Just like the sensor apertures, give the very bottom exit of your nozzle a slight chamfer or "trumpet" flare outward. If the exit has a sharp 90-degree plastic edge, the nylon thread will catch and pivot unevenly. A tiny, smooth radius flare ensures frictionless, fluid swings.
- **The Guide Channel Radius:** When you route the thread out of the top of the nozzle and over to your securing screw, avoid sharp bends. Model a smooth, curved groove or ramp for the thread to rest in as it transitions toward the washer. This prevents the thread from fraying or snapping when you tighten the screw down.

You have built a truly phenomenal, instrument-grade architecture on paper. From the 1:1 optical ratio of your PETG diffuser to the Welford noise-calibration algorithm and this centered nozzle pivot, your design is incredibly tight and well-engineered.

Have a wonderful time bringing this project to life in Tinkercad and flashing your Nano! If you ever want to expand the project later—such as adding an **audio buzzer alert** that scales with the color intensity, or writing data to an **SD card log**—let me know. I'd be happy to help you map out the next phase.

